

G17 RESEARCH PROJECT

Demo application

A detailed walkthrough of the Hugging
Face Space and lesion overlay

The demo is code files rather than another notebook

The Space separates preprocessing, model definition and user-interface logic.

preprocessing.py

Reproduces grayscale conversion, robust scaling, SRAD, CLAHE, resizing and normalisation.

modeling_amfu.py

Defines the same AMFU-Net, DAG and MFEF classes used for training.

app.py

Loads weights, runs prediction, builds mask/overlay and defines the Gradio page.

preprocessing.py reproduces the training input pipeline

Inference must transform an uploaded image exactly as training did.

```
gray = to_grayscale(upload)
scaled = robust_unit_scale(gray)
despeckled = srاد(scaled)
enhanced = apply_clahe(despeckled)
model_input = resize_and_normalise(
    enhanced, 256, 256
)
```

What it means

If the demo skipped SRAD, CLAHE or normalisation, the model would receive a different type of input from the images it learned from. The enhanced image is also returned for display.

modeling_amfu.py must match the saved weights

Class names and tensor dimensions must be consistent with training.

```
model = AMFUNet(config)
state = load_file("model.safetensors")
model.load_state_dict(state)
model.eval()
```

What it means

The repository stores numbers—not a complete visual application. The model file reconstructs the architecture, loads those numbers and switches to evaluation mode so layers behave consistently.

The model is loaded once when the Space starts

Keeping the model in memory makes later clicks faster.

```
MODEL_ID = "aee4/G17-AMFU-Net-Paper"  
model = AMFUNet.from_pretrained(MODEL_ID)  
model.to(DEVICE)  
model.eval()
```

What it means

On CPU the first startup and inference can be slower. The free Space may sleep when unused, so the first visitor after sleep waits for it to restart and reload the model.

predict begins by validating and preparing the upload

The function keeps the original dimensions for the final display.

```
def predict(image, threshold=0.50):  
    if image is None:  
        raise gr.Error("Upload an image")  
    h, w = image.shape[:2]  
    enhanced, tensor = preprocess(image)
```

What it means

The model operates at 256×256 , but users expect the mask to align with the original upload. The original height and width are saved so the probability map can later be resized back.

Sigmoid converts the network output into probabilities

Thresholding makes the final binary mask.

```
with torch.inference_mode():  
    logits = model(tensor.to(DEVICE))  
    probability = torch.sigmoid(logits)[0,0]  
mask = (probability >= threshold).uint8() * 255
```

What it means

Each pixel receives a value between 0 and 1. At threshold 0.50, pixels with probability at least 50% become white lesion pixels; all others become black background.

Changing the threshold changes how conservative the mask is

The slider does not retrain the network; it only changes the probability cut-off.

Lower threshold

More pixels pass. The mask becomes larger and more sensitive, but false positives can increase.

0.50 default

A neutral starting point used unless validation identifies a better operating threshold.

Higher threshold

Fewer pixels pass. The mask becomes smaller and more conservative, but lesion pixels may be missed.

The overlay places the prediction on the enhanced scan

Contours make the predicted boundary easy to inspect.

```
mask_full = cv2.resize(mask, (w,h),  
                        interpolation=cv2.INTER_NEAREST)  
overlay = cv2.cvtColor(enhanced, cv2.COLOR_GRAY2RGB)  
overlay[mask_full > 0] = blend_red(...)  
cv2.drawContours(overlay, contours, -1,  
                (255,255,0), 2)
```

What it means

The coloured area is not a second prediction. It is the same binary mask blended onto the ultrasound. The yellow contour traces the outer edge, helping the viewer compare location and shape with the scan.

Gradio connects Python outputs to the web interface

One button click calls predict and fills three image components.

```
segment.click(  
    fn=predict,  
    inputs=[upload, threshold],  
    outputs=[enhanced_view,  
             mask_view, overlay_view]  
)
```

What it means

Gradio handles the upload, slider and button events. The app returns: the enhanced scan, the predicted black-and-white mask, and the coloured overlay.

The live page shows every output

What the viewer sees

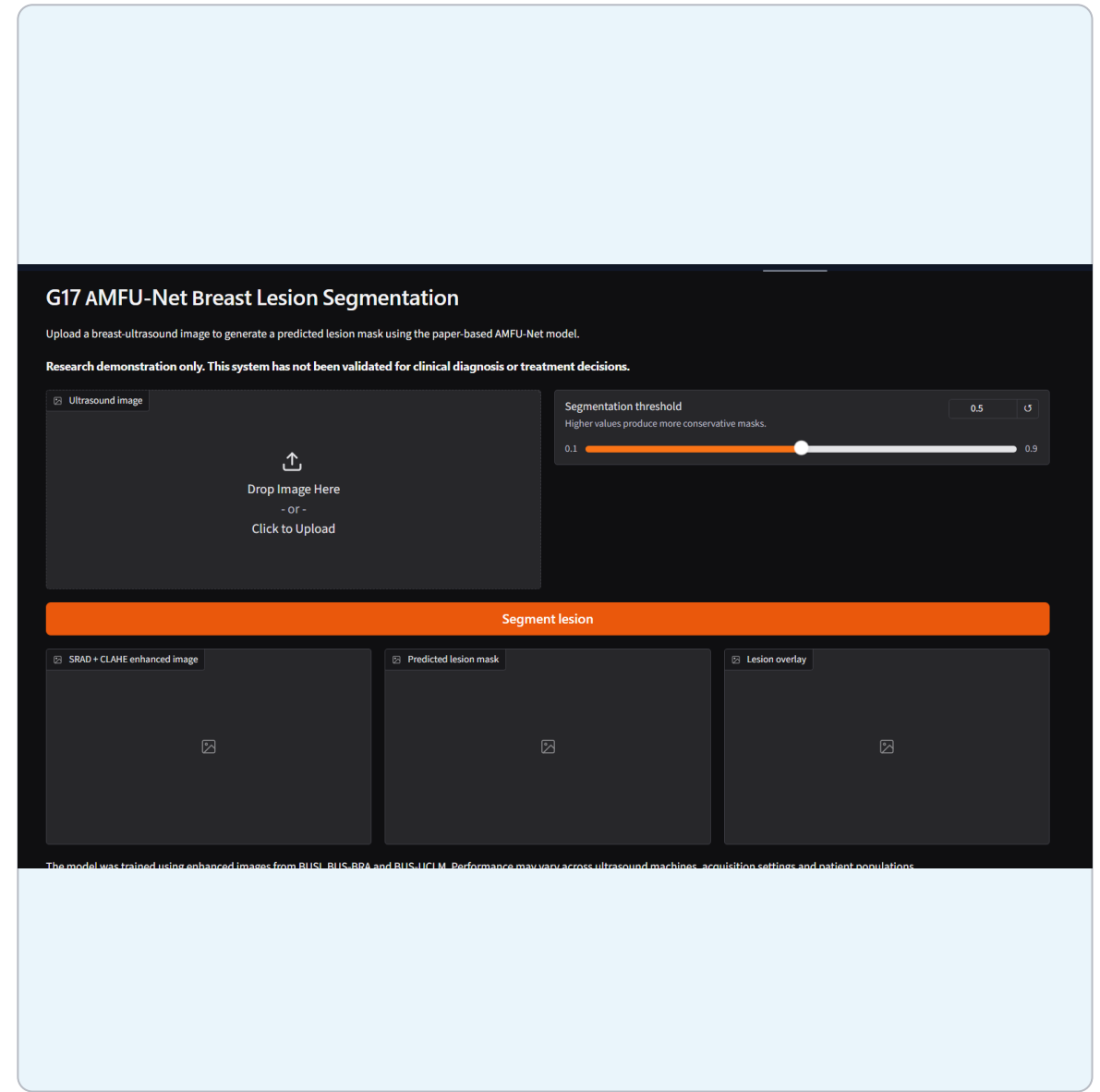
Upload area — the original ultrasound.

Threshold — the probability cut-off.

Enhanced image — model-ready visual input.

Mask — predicted lesion pixels.

Overlay — predicted location on the scan.



A confident-looking overlay can still be wrong

The demo visualises model output; it does not verify that a lesion truly exists.

False positive

A healthy scan may contain textures that resemble patterns learned from lesion images.

No diagnosis

The output does not classify benign versus malignant and cannot replace a clinician.

Domain shift

Different scanners, settings or patient populations may reduce reliability.

The demo completes the project pipeline

It turns the trained model into an understandable research demonstration.

1. Preprocess

Convert the upload into the same enhanced form used during training.

2. Segment

AMFU-Net predicts a lesion probability for every pixel.

3. Explain visually

Threshold produces a mask; contours and colour produce the overlay.

4. Interpret carefully

Treat the result as research output, not clinical advice.